Dipartimento di Ingegneria Informatica, Automatica
e Gestionale

Parameter-Efficient Few-Step Distillation of Diffusion Models via LoRA

Computer Vision — Project 5

Lightweight Diffusion Models for Resource-Constrained Environments

Professor:
Irene Amerini

Students:
Claudia Giuseppini — 2069387
Benedetta Conti — 2045621
Fabio Falconi — 2048079

Academic Year 2025/2026

Contents

1 Introduction

Denoising Diffusion Probabilistic Models (DDPMs) [1] generate images by learning to gradually remove noise. They produce high-quality results, but generating a single image requires running the network many times in sequence (up to one thousand times), which makes them slow and expensive. This is a limitation when we want to run them on small devices or train them with limited hardware.

This project implements a standard DDPM baseline on CIFAR-10 and then integrates a distillation designed to make sampling faster. We evaluate the trade-off between computational efficiency (sampling time) and generation quality (FID, Inception Score), and we also measure sample diversity (Precision/Recall).

A faster deterministic sampler called DDIM [2] can reduce the number of steps, but quality collapses once the steps become very few. The reason is not that the network predicts noise badly at those steps: it is the *discretization error* that accumulates when we take large jumps along the denoising path. The distillation method we implemented mitigates exactly this error, while keeping the extra training cheap:

> **Parameter-efficient few-step distillation via LoRA.** We freeze the trained DDPM parameters (the teacher model) and train only small low-rank adapters [4] inserted into the attention layers (the student model) with a different objective [3]. The student learns to reproduce, in very few steps, what the teacher produces with many DDIM sub-steps.

Because only the adapters are trained (about 0.1% of the weights) and they can be merged back into the network at inference, the method reduces inference time at the same matched quality compared to DDIM models, without increasing the training time substantially.

2 Related Works and known theoretical Limitations

- **DDPM** [1] : will be our baseline generative model as state of the art. *Limitation:* it usually has very slow sampling ($T = 1000$ sequential network evaluations).
- **DDIM** [2] : is a deterministic sampler that allows fewer steps with the base model, usually applied to compensate DDPM limitation. *Limitation:* quality collapses under high step reduction.
- **Distillation** [3] : distils a teacher into a student that reduces the number of steps. *Limitation:* it re-trains the whole network at each stage, which is costly.
- **LoRA** [4] : low-rank adapters for cheap and efficient adaptation of a frozen model.

DDPM and DDIM reppresent the state of the art, as seen in the references. LoRa is an improvement of the previous works: it bring distillation with trajectory matching to diffusion. This addresses the cost limitation of distillation while reducing the discretization-error limitation of few-step DDIM.

3 Experimental Setup

This section lists the configuration of our experiments.

Data. CIFAR-10: 60,000 colour images of size 32×32 with 3 channels (RGB), split into 50,000 for training and 10,000 for testing, across 10 classes. Pixel values are rescaled from $[0, 1]$ to $[-1, +1]$ so that the data is centred on zero, which matches the zero-centred Gaussian noise used by the diffusion process.

Model and training. An attention U-Net with `base_channels`= 64 and `time_dim`= 256 (15.3$M parameters). We use $T = 1000$ diffusion steps with a linear noise schedule, the AdamW optimizer, batch size 128, and 80 training epochs.

Distillation. For each target step budget $K \in \{8, 4, 2, 1\}$ we train one student for 10 epochs. Only the LoRA adapters are trained: 16,384 parameters out of 15.3M, about 0.107% of the model.

Metrics. We report FID and KID (image fidelity), Inception Score (IS, sharpness/recognisability), Precision and Recall [5] (fidelity vs. diversity), and seconds per image (speed). All metrics are computed on 10 000 generated images, compared against the CIFAR-10 test set with the `torch-fidelity` library, and every method starts from the same seeded initial noise so the comparisons are fair.

Hardware and reproducibility. Experiments run on a single GPU (NVIDIA T4 on Kaggle). We fix all random seeds (seed = 42), enable deterministic algorithms, seed the data-loader workers and the sampling noise generator, and export the resolved configuration and the exact package versions.

4 Method

We first explain the standard DDPM model following the standard from the literature, then the engineering optimizations we added around it, and finally the distillation.

4.1 How diffusion models work

A diffusion model is built on two opposite processes. The *forward* process takes a real image and adds a little random noise, step after step, until after $T = 1000$ steps the image is pure noise. This process is fixed and requires no learning. The *reverse* process is where a neural network learns to remove a little noise at a time. Once trained, we can start from pure noise and apply the network repeatedly to "uncover" a new image. In short, generating an image means denoising repeatedly.

The reason we use so many small steps is that each single step is easy: from a slightly noisy image to a slightly-less-noisy one, the change is tiny and the network can predict it accurately. Asking the network to jump from pure noise to a finished image in one shot would be almost impossible, because many different images could explain the same noise, and the average answer is blurry.

4.2 The forward noising process

The forward process is described by a single formula that directly gives the noisy image at any level t, without simulating all the intermediate steps. The closed formula is possible because the diffusion is modeled like a Markov process where every state depends entirely only on the previous state. Therefore we can predict the n-th state directly if we have the first one. The closed formula follows:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I). \tag{1}$$

The noisy image x_t is a weighted mix of two things: the clean image x_0 and pure random noise ε. The weight $\bar{\alpha}_t$ ("alpha-bar at t") is a number between roughly 1 and 0 that decides how much of each we keep. At small t, $\bar{\alpha}_t$ is close to 1, so the image dominates and there is little noise; at large t, it approaches 0, so noise dominates. The value $\bar{\alpha}_t = \prod_{s=1}^{t}(1 - \beta_s)$ is just the running

product of the "keep" factors, where each β_s says how much noise is added at step s. We use a *linear schedule*, meaning the β_s grow linearly from a small start value to a larger end value.

4.3 The standard denoising network: an attention U-Net

The network that removes noise is a **U-Net** [1]. Its name comes from its shape that is like the letter U: on the way down it *compresses* the image (reduces the spatial size and increases the number of channels, capturing the overall context); on the way up it *expands* it back to the original size (rebuilding the fine detail). "Channels" here are the internal feature maps the network works with; more channels means more capacity.

Input and output. The input is a noisy image ($3 \times 32 \times 32$) together with the noise level t; the output is the network's estimate of the noise ε, with the same shape as the image. Predicting the *noise* (rather than the clean image) is the standard formulation. it lets us predict the next image on the chain "noisy -> clear" if we know the noise difference to the next state.

Skip connections. Shortcuts link the matching levels of the down and up paths, so the fine details captured before compression (during down path) are not lost when the image is rebuilt (during up path).

Channel multipliers. The width grows as we go deeper, controlled by the multipliers `channel_mults` (e.g. $64 \to 128 \to 128$).

Residual blocks (ResBlocks). The basic building block used at every level of the network. Besides its normal processing, each block also carries its input directly forward and adds it back to the output (a "residual" shortcut). This lets the block learn only the *change* to apply, rather than rebuilding the signal from scratch, which keeps information from getting lost and makes deep networks much easier and more stable to train.

Normalization (GroupNorm) and activation (SiLU). After each block we *normalise* the internal values: rescaling them to a consistent range so that no layer produces numbers that are too large or too small, which keeps training stable. We use Group Normalization, a variant that works well even with small batches and generative tasks. We then apply SiLU (also called Swish), the non-linear function that lets the network learn complex relationships. Both are standard, well-tested choices for diffusion U-Nets.

Down- and up-sampling. Operations that change the image resolution as it moves through the U: on the way down we shrink it (fewer pixels) so the network can capture the overall structure cheaply, and on the way up we enlarge it back to full size to restore the fine detail.

Time embedding. The network must behave differently depending on the noise level t to correctly predict the next step. We turn the single number t (from 0 to 999) into a vector of `time_dim`$= 256$ numbers using sine and cosine functions at several frequencies (a *sinusoidal timestep embedding*), and we feed this vector into the layers. This tells the network "how much noise to expect", so a single network can handle all 1000 levels.

Self-attention. Convolutions only look at nearby pixels, but good denoising also needs coordination (the whole image should agree on what it is becoming). *Self-attention* lets every position in the image look at every other position and gather information from anywhere. Technically, for each position the network builds three vectors: a *Query* ("what am I looking for"), a *Key* ("what do I offer") and a *Value* ("the information I carry"). Then it compares each Query with all Keys to decide how relevant every other position is, and then takes a weighted average of the Values. We use *multi-head* attention (several comparisons in

parallel, each able to focus on a different kind of relation) and we insert it only at the lower resolutions, because comparing every position with every other costs grow exponentially with the number of positions, so it is cheap only after the image has been compressed. Importantly, the Query/Key/Value/Output projections of these attention blocks are exactly where we later insert the LoRA adapters used by our novelty.

4.4 The training objective

For training, we take a real image, pick a random noise level t, add the corresponding amount of noise using Equation 1, and ask the network to guess the noise we added. The loss is the mean squared error (MSE) between the true noise and the predicted noise:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{x_0,\, t,\, \varepsilon}\big\|\varepsilon - \varepsilon_\theta(x_t, t)\big\|_2^2. \tag{2}$$

ε is the real noise we added, and $\varepsilon_\theta(x_t, t)$ is the network's guess. The double bars with subscript 2 and power 2 mean "sum of the squared differences". The symbol $\mathbb{E}$ (expectation) just means we average this error over many images, many noise levels, and many random noises. Minimising it teaches the network to denoise at every level. We will see that this loss drops quickly and then flattens, because most noise levels are "easy"; a low loss therefore does not by itself guarantee good images, so we always judge quality with FID.

4.5 Engineering optimizations we added around the baseline

The items above are the standard DDPM. However, we saw that with the resources given to us by Kaggle and Colab, we couldn't produce images with high quality, because inserting the necessary parameters (e.g. higher nummber of channels) required a lot more computational power than we had.

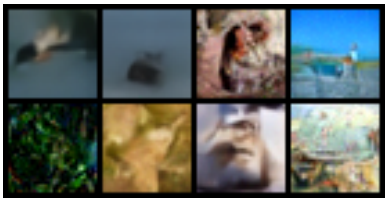

Figure 1: sample from strong profile, DDIM 50

Figure 2: Sample from ultra profile, DDIM 50

We can see that the ultra profile produces slightly clearer images than strong, meaning that increasing the profile parameters yealds better results and our limited resolutions is dictated by limited hardware.

Therefore, we decided to add a few optimizations to make training more stable, faster, fully reproducible, and have better quality with less time spent on the resources we had. They produce a solid infrastructure for the baseline.

EMA (Exponential Moving Average). During training we keep a second, "smoothed" copy of the weights, updated as $W_{\text{ema}} \leftarrow \text{decay} \cdot W_{\text{ema}} + (1 - \text{decay}) \cdot W$ with *decay* close to 1. Because the optimizer makes the raw weights wobble around a good configuration, this time-average is more stable and produces noticeably better images at no extra cost. We generate from the EMA weights, and the EMA model is also the *teacher* for distillation. On short runs a warm-up (scale the decay to not give early weights a disproportionate relevance) on the decay avoids the average staying fixed to the random initialization.

Mixed precision (AMP). Some computations are done in reduced (16-bit) precision with `torch.cuda.amp`, which makes training faster and lighter on memory with no visible loss of quality. A *GradScaler* rescales the gradients to avoid the numerical problems that reduced precision can cause.

Data augmentation. We apply a random horizontal flip to the training images. A mirrored CIFAR image is still a plausible image, so this enriches the data without distorting its distribution, unlike crops or rotations, which would generate unnatural images and worsen the FID.

Gradient clipping. We cap the size (norm) of the gradients before each update (that uses AdamW for optimization), so an occasional huge gradient cannot destabilise training (during loss minimization).

Determinism and reproducibility. We fix the seed for all random sources (Python, NumPy, PyTorch, CUDA), enable deterministic algorithms (`use_deterministic_algorithms`, `cudnn.deterministic`), and seed the data-loader workers, so re-running the code gives the same results.

Seeded shared noise bank. For evaluation we fix the initial random noise with a seeded generator, so every sampler denoises the same starting noise. This makes all comparisons fair: any difference in the output is due to the method, not to lucky noise.

Checkpointing and resume. We save the weights periodically (`ddpm_latest`) and at the end (`ddpm_final`, `ddpm_ema`), and each distilled student can resume from its own checkpoint.

4.6 Fast sampling with DDIM

Generating with the full DDPM means applying the network 1000 times. The standard to fasten things is DDIM. DDIM speeds this up by *skipping* levels in a deterministic way. One DDIM step from level t to a later level t' is:

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}, \qquad x_{t'} = \sqrt{\bar{\alpha}_{t'}}\,\hat{x}_0 + \sqrt{1-\bar{\alpha}_{t'}}\,\varepsilon_\theta(x_t, t). \tag{3}$$

First we use the predicted noise to estimate the clean image $\hat{x}_0$ (this is just Equation 1 solved for x_0). Then we re-noise that estimate down to the next level t'. The setting $\eta = 0$ makes the process deterministic (no extra random noise is added along the way). With fewer, larger jumps this is much faster, but each large jump introduces a small approximation error, and these errors add up, which is why very few steps hurt quality.

4.7 LoRA few-step distillation

What LoRA is. We said that DDPMs take a lot of time to make all 1000 steps for the generation of one image. DDIMs are one way to speed things up, but can hurt quality with few steps. So we decided to implement LoRA to distill a new model more capable.

Inside the attention layers, the network multiplies its input by large weight matrices W. Retraining all of them would be expensive. LoRA freezes W and adds next to it two *small* matrices A and B, whose product $B{\cdot}A$ represents the correction ΔW we want. Only A and B are trained. The number of intermediate channels in A and B is the *rank* (e.g. 4): a small rank means very few trainable numbers. During the forward pass the layer computes $y = Wx + (BA)x$: the frozen original plus a learned correction. We start with $B = 0$, so at the beginning the student behaves exactly like the teacher. At the end, A and B can be merged into W ($W' = W + BA$), so the model has the same shape as the original and takes less inference time at matched quality.

The distillation objective. The trained baseline (EMA weights) is the frozen teacher ε_ϕ; the student ε_θ is the same network with LoRA adapters. For a target step budget K we build a short schedule and pick a random interval $(t \to t')$. We form the starting state x_t by forward-diffusing a real image (with Equation 1). The *teacher* refines x_t from t to t' using m small DDIM sub-steps, giving a good target $x_{t'}^{\text{teacher}}$; the *student* must reach the same point in a *single* DDIM step, giving $x_{t'}^{\text{student}}$. The loss is:

$$\mathcal{L}_{\text{distill}} = \left\| x_{t'}^{\text{student}} - x_{t'}^{\text{teacher}} \right\|_2^2 \; + \; \lambda \left\| \varepsilon_\theta(x_t, t) - \varepsilon_\phi(x_t, t) \right\|_2^2. \tag{4}$$

The first term forces the student's one-step jump to land where the teacher's careful multi-step path lands: this is what teaches the student to compensate the error of large jumps. The second term (weighted by λ) is an auxiliary "noise-consistency" term that keeps the student's noise prediction close to the teacher's, which stabilises training. We train one student per target $K \in \{8, 4, 2, 1\}$, updating only the LoRA adapters.

Why this is efficient. LoRA does not make each denoising step faster. The speed-up comes from using fewer steps. LoRA's role is to make the adaptation cheap to train (only $\sim$0.1% of the parameters, so it fits on a single GPU) and free at inference (the adapters merge into the weights). So we get few-step sampling without the quality collapse of plain DDIM, and without the cost of full-model distillation.

Models created. We decided to produce many different models from LoRA, to determine the most important parameters that yield better results. Therefore, the models analyzed in the results will have some differences. We will try models with different objectives (loss functions) to understand if LoRA works as intended because of its objective, with different ranks for the matrices A and B, and finally with different sources for the training images (in one case images will be pure random noise, in the other they will be the result of the forward noising process from a real image).

5 Results

5.1 Baseline training

Figure 3 shows the training loss decreasing from about 0.089 to 0.032 and then flattening, as expected. We noticed from the figure that beyond epoch 40 the loss stabilizes, showing that we can reduce the number of epochs and the training time without hurting the quality. This was also clear from previous experiments where we tried to slowly increase only epochs to increase quality. Figures 44, 5 and 6 show uncurated samples from the baseline: the global structure (animals, vehicles, backgrounds) is visible, though fine detail is limited. This consistent with a compact model trained under a single-GPU budget.

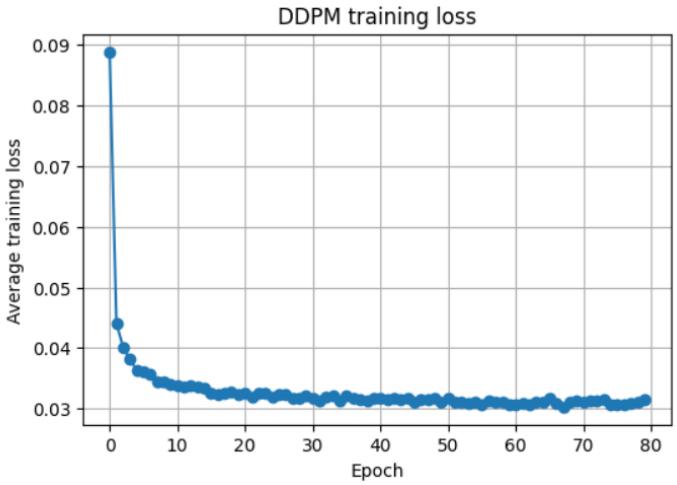

Figure 3: Baseline DDPM training loss over 80 epochs.

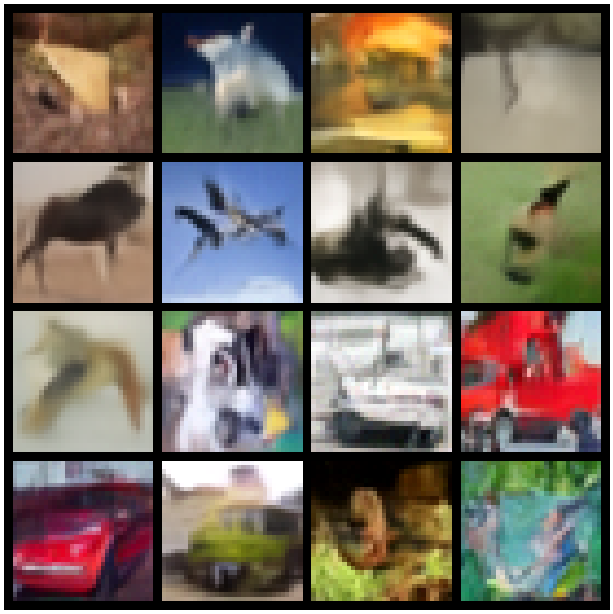

Figure 4: Uncurated baseline samples (DDIM, 20 steps).

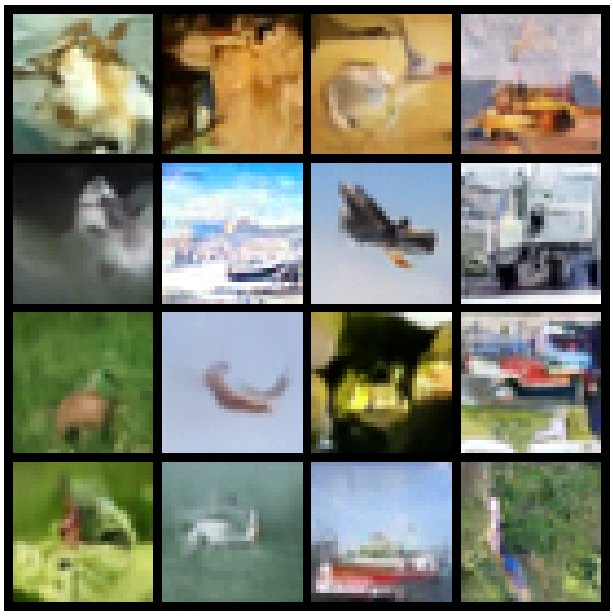

Figure 5: Uncurated baseline samples (DDIM, 50 steps).

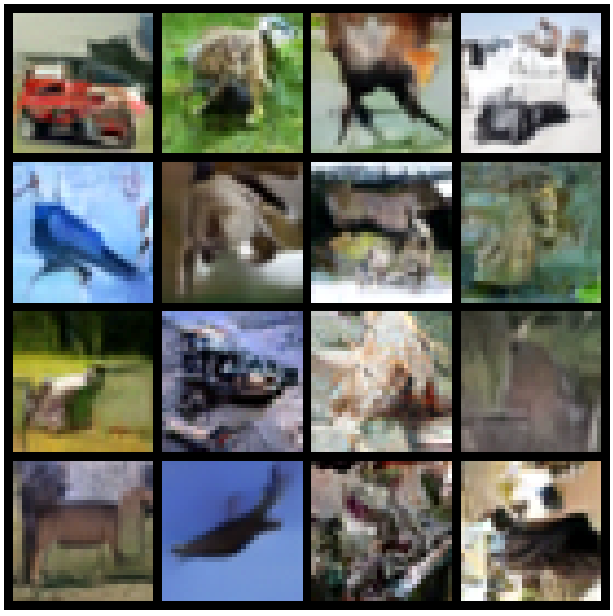

Figure 6: Uncurated baseline samples (DDIM, 100 steps).

5.2 Quality and speed vs. number of steps

Table 1 reports the baseline across step budgets. Fewer steps means faster but lower-quality sampling. In particular, **recall (diversity) collapses** long before precision (Figure 8).

Table 1: Baseline: DDIM at different step counts (FID on 10 000 images). The DDPM row uses 1000 images and is reported only for speed-up factors.

Sampler	Steps	FID ↓	IS ↑	Precision	Recall	s/img	Speed-up†
DDPM (ref)	1000	67.3*	6.26	0.847	0.381	0.674	1×
DDIM	100	29.33	7.23	0.751	0.326	0.052	12.8×
DDIM	50	29.79	7.32	0.777	0.306	0.026	25.2×
DDIM	20	32.43	7.32	0.834	0.243	0.011	60.5×
DDIM	10	40.89	7.02	0.893	0.141	0.006	112.9×
DDIM	8	47.12	6.83	0.903	0.102	0.005	136.5×
DDIM	4	103.28	5.29	0.853	0.015	0.003	242.4×
DDIM	2	188.84	3.73	0.401	0.001	0.002	∼360×
DDIM	1	325.11	1.56	0.005	0.000	0.001	∼510×

† vs. DDPM-1000. * On 1000 images (noisier).

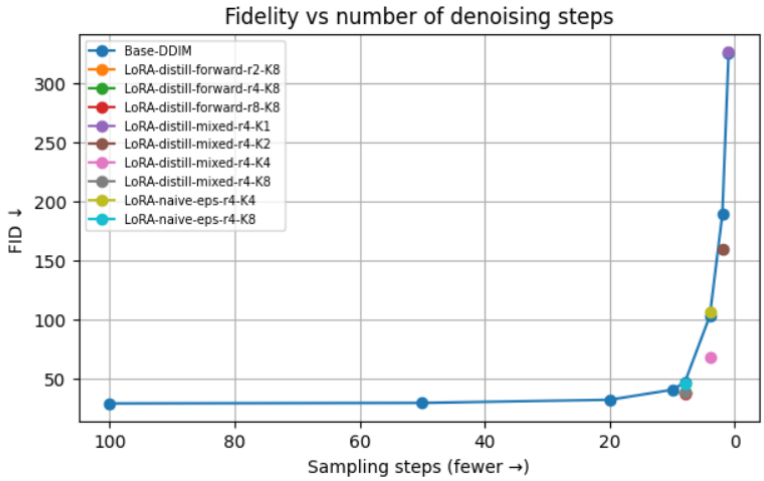

Figure 7: FID vs. number of denoising steps. DDIM (blue curve) degrades sharply below ∼10 steps; the distilled LoRA students sit below the DDIM curve at the same step count.

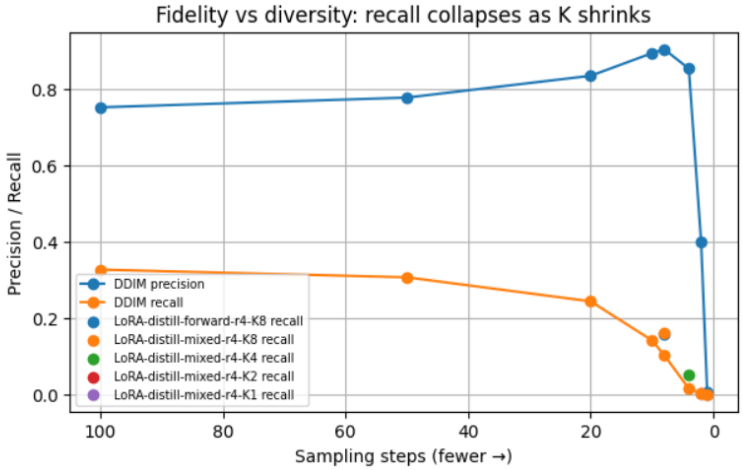

Figure 8: Fidelity vs. diversity. As steps shrink, precision stays high while recall collapses: few-step sampling trades diversity for apparent fidelity.

Fidelity vs. Diversity. Reducing the number of steps has two opposite effects that a single score would blur together. As steps drop from 100 to 8,precision even rises ($0.75 \rightarrow 0.90$): the individual images stay realistic, and if anything look cleaner. At the same time recall drops ($0.33 \rightarrow 0.10$, and to 0 at 4 steps and below): the model covers a smaller and smaller part of the real data. In other words, few-step sampling does not make each image worse, it makes the model generate less variety, drifting toward a narrow set of similar but more precise outputs. FID mixes quality and variety into one number and would hide this; Precision and Recall separate them and reveal that the real casualty of high step reduction is diversity, not fidelity.

5.3 LoRA training

Before comparing the generated samples against plain DDIM, we first inspect the LoRA training itself. In every distilled student the base DDPM weights are frozen and only the LoRA adapters are updated: for the rank-4 configuration this means 16 384 trainable parameters out of about 15 302 147, roughly 0.107% of the model. Any change in the student's behaviour therefore comes from a very small low-rank correction, not from retraining the full denoising network.

Training behaviour (loss). Table 2 reports the distillation loss of the main students, and Figure 9 shows the corresponding curves. In all cases the loss is stable and does not diverge, which confirms that the adapters learn a consistent correction on top of the frozen DDPM. The absolute loss value, however, depends strongly on the step budget K: it is around 10^{-5} at $K = 8$, rises to 10^{-4} at $K = 4$, and becomes much larger at $K = 2$ and especially $K = 1$. This makes sense: the smaller K is, the longer the part of the teacher trajectory that a single student step must approximate, so the mapping is harder to learn with a tiny low-rank adapter. Also, the loss is dominated by the x_{next} trajectory-matching term rather than by the auxiliary noise-prediction term, which confirms that the student is trained to imitate the teacher's next state, not to re-learn the standard DDPM objective.

Table 2: Training behaviour of the main distilled LoRA students (rank-4 adapters, 16 384 trainable parameters, 0.107% of the model).

Student	State source	K	Initial loss	Final loss	Final x_{next} loss
distill-forward-r4-K8	forward only	8	3.4×10^{-5}	2.4×10^{-5}	1.7×10^{-5}
distill-mixed-r4-K8	mixed states	8	3.2×10^{-5}	2.8×10^{-5}	2.0×10^{-5}
distill-mixed-r4-K4	mixed states	4	4.01×10^{-4}	2.74×10^{-4}	2.57×10^{-4}
distill-mixed-r4-K2	mixed states	2	1.94×10^{-2}	1.71×10^{-2}	1.71×10^{-2}
distill-mixed-r4-K1	mixed states	1	8.13×10^{-1}	7.70×10^{-1}	7.70×10^{-1}

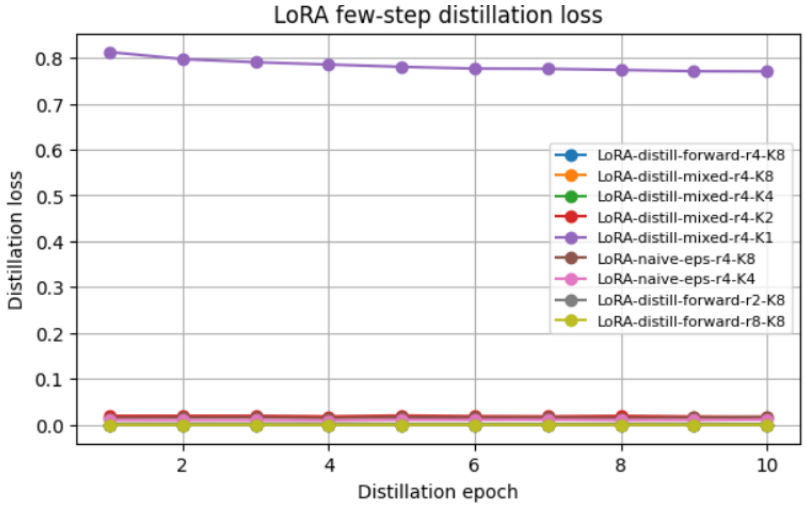

Figure 9: Distillation loss over epochs for the different LoRA students. The $K = 1$ student (top curve) stays high, while the $K \geq 4$ students converge to very low values.

Full evaluation. A low training loss does not prove that a student produces good images, so Table 3 reports the full generation metrics (FID, KID, IS, Precision, Recall) for all LoRA students, evaluated on 10 000 images. All students at a given K have essentially the same sampling time as plain DDIM at that K, confirming that the LoRA adapters add no inference overhead for the same steps, but to obtain the same quality they need less steps/inference time.

Table 3: Complete evaluation of all LoRA students (10 000 images). "Role" indicates whether the row is a main student, a rank/state ablation, or a naive-eps control.

Student	Role	K	FID ↓	KID ↓	IS ↑	Prec.	Recall
Distillation objective							
distill r2, forward	rank ablation	8	38.32	0.0195	7.12	0.871	0.152
distill r4, forward	main (K=8)	8	38.48	0.0195	7.14	0.870	0.157
distill r8, forward	rank ablation	8	**37.67**	0.0190	7.17	0.863	**0.162**
distill r4, mixed	state ablation	8	38.24	0.0194	7.13	0.871	0.159
distill r4, mixed	main (K=4)	4	68.34	0.0408	6.18	0.848	0.051
distill r4, mixed	extreme	2	159.12	0.1298	3.85	0.451	0.002
distill r4, mixed	extreme	1	326.57	0.3449	1.55	0.006	0.000
Naive eps-loss (control)							
naive-eps r4	control	8	46.16	0.0261	6.76	0.914	0.102
naive-eps r4	control	4	106.79	0.0804	4.90	0.869	0.014

At $K = 8$ all distilled students land around FID 38 regardless of rank (r2/r4/r8 differ by less than one FID point), which reinforces the parameter-efficiency claim: even rank 2 is enough. The forward and mixed state sources are also essentially tied (38.48 vs. 38.24), so conditioning on teacher-trajectory states does not help, and we do not rely on it. The naive-eps controls are decisively worse than their distilled counterparts (46.16 vs. 38.24 at $K = 8$, and 106.79 vs. 68.34 at $K = 4$), which proves the distillation objective is the cause of the improvement. Finally, the $K = 1$ student, whose loss remained high in Table 2, also fails on the metrics (FID 326): a single denoising step is not enough. The $K = 8$ and $K = 4$ are therefore the meaningful ones, where the adapter has clearly learned the teacher correction.

5.4 Matched-step comparison

Table 4 compares plain DDIM-K against the distilled LoRA-K at the same step budget, same initial noise, and same base weights. The only difference is the LoRA adapter.

Table 4: Matched step budget K: plain DDIM vs. distilled LoRA.

K	DDIM FID	Distilled FID	ΔFID	DDIM Recall	Distilled Recall	Time ratio
8	47.12	**38.24**	**+8.9**	0.102	0.159	1.01×
4	103.28	**68.34**	**+34.9**	0.015	0.051	1.01×
2	188.84	**159.12**	**+29.7**	0.001	0.002	1.01×
1	325.11	326.57	−1.5	0.000	0.000	1.00×

The distilled student improves FID at $K = 8, 4, 2$, with the largest gain at $K = 4$ (+34.9), and partially **restores diversity** (recall +0.057 at $K = 8$). At $K = 1$ both models fail. Inference cost is unchanged for almost every model listed. The distilled $K = 8$ model is, however, 10.6× faster than DDIM-100 and the $K = 4$ model 18.9× faster, at a fraction of the quality loss of plain DDIM at the same step count.

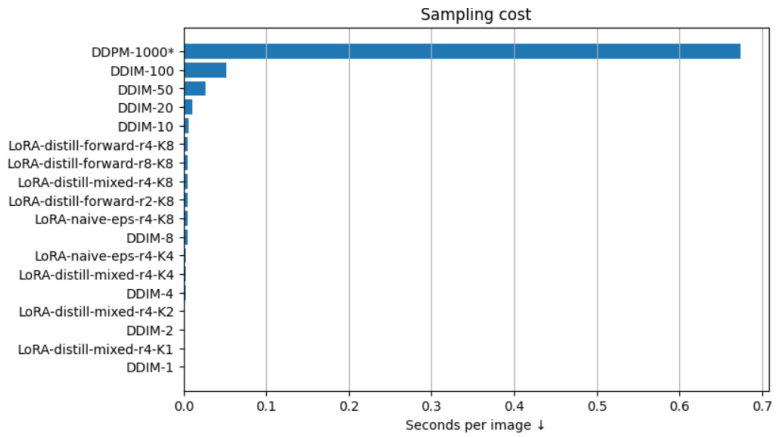

Figure 10: Compared sampling time costs for all the models experienced.

As we can see from figure 9, inference time does not change substantially between same step models, but table 2 shows that quality improves. Therefore we can conclude that to produce the same quality image, LoRA reduces inference time. We can see this visually in the images comparison below.

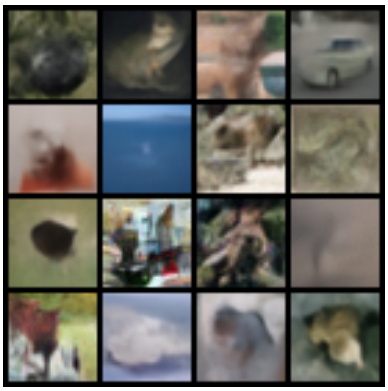

Figure 11: DDIM 8 steps

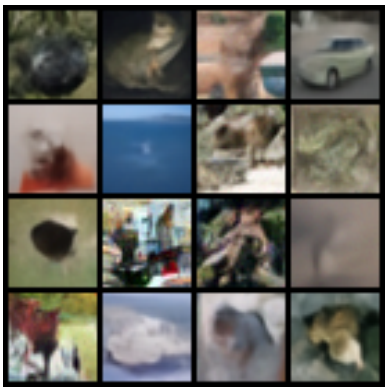

Figure 12: LoRA 8 steps

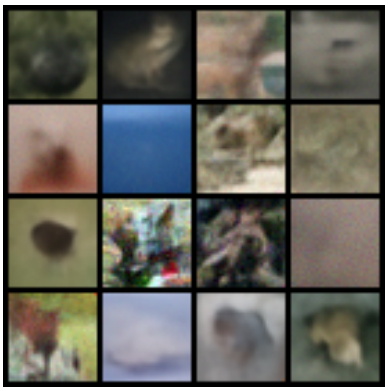

Figure 13: DDIM 4 steps

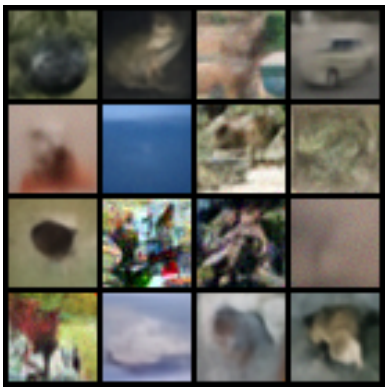

Figure 14: LoRA 4 steps

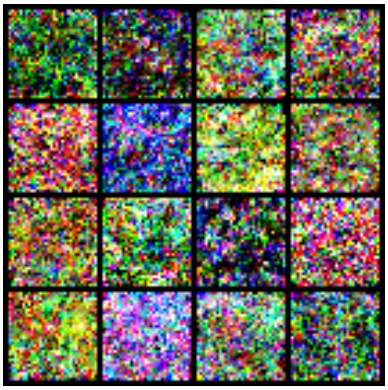

Figure 15: DDIM 1 step

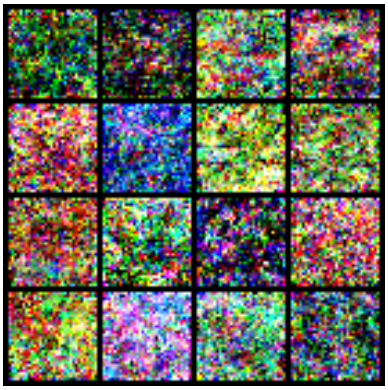

Figure 16: LoRA 1 step

5.5 Ablation 1: distillation objective vs. naive eps-loss

To prove that the gain comes from the distillation objective (losss function) and not simply from the extra LoRA parameters, we train an identical LoRA (same rank, same trainable parameters) with the standard noise-prediction loss (Equation 2) at low steps.

Table 5: naive eps-loss LoRA vs. distillation, matched K and rank.

K	Naive-eps LoRA ΔFID	Distillation ΔFID
8	+0.96	**+8.9**
4	−3.51 (worse)	**+34.9**

The naive control gives ≈0 improvement at $K = 8$ and is worse than plain DDIM at $K = 4$, while distillation gives large gains. This isolates the trajectory-matching objective as the direct cause of the improvement.

5.6 Ablation 2: LoRA rank ($K = 8$)

FID: *rank*=2: 38.32, *rak*=4: 38.48, *rank*=8: 37.67.

Rank barely matters: even rank 2 captures almost all of the benefit, reinforcing the parameter-efficient claim (only ∼0.1% of the weights are trained for basically all Ks).

5.7 Ablation 3: distillation state source ($K = 8$)

FID for Forward-diffused states only: 38.48

FID for mixing teacher-trajectory states ($p = 0.5$): 38.24
They are essentially tied. Conditioning on generated trajectory states does *not* help; forward-diffusion states are sufficient. We report this as a negative result and do not rely on trajectory-state mixing in the final method.

6 Observations

- **Weak absolute baseline.** Under the single-GPU budget the baseline reaches FID ≈ 29; samples show global structure but limited detail. This is a hardware-driven limitation, but because all comparisons are relative to the same teacher, the conclusions about the methods hold regardless of absolute quality.
- **Breakdown at $K = 1$.** The method does not recover single-step sampling on this baseline.
- **FID variability with sample size.** FID depends slightly on how many images are used to compute it, so it carries some noise. We therefore evaluate every comparison on the same 10 000 images. The K=4K=4 K=4 improvement (+34.9+34.9 +34.9) is far larger than this noise and is clearly reliable; the K=8K=8 K=8 improvement (+8.9+8.9 +8.9) is smaller, and confirming it firmly would require repeating the experiment with several random seeds.

7 Conclusion

We presented a parameter-efficient few-step distillation of a DDPM using LoRA adapters and a trajectory-matching objective. On CIFAR-10 the method improves few-step generation quality at a matched step budget, partially restores the diversity lost by high step reduction, and adds no inference overhead, yielding $\sim$10–19$\times$ speed-ups over the DDIM-100 baseline. A naive-eps control confirms that the distillation objective (not the extra parameters) is responsible for the gain.

References

[1] J. Ho, A. Jain, P. Abbeel. *Denoising Diffusion Probabilistic Models.* NeurIPS 2020.

[2] J. Song, C. Meng, S. Ermon. *Denoising Diffusion Implicit Models.* ICLR 2021.

[3] T. Salimans, J. Ho. *Progressive Distillation for Fast Sampling of Diffusion Models.* ICLR 2022.

[4] E. Hu, Y. Shen, P. Wallis, et al. *LoRA: Low-Rank Adaptation of Large Language Models.* 2021.

[5] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, T. Aila. *Improved Precision and Recall Metric for Assessing Generative Models.* NeurIPS 2019.